

< 200 CONCRETE OPEN PROBLEMS IN MECHANISTIC INTERPRETABILITY >

200 COP in MI: Techniques, Tooling and Automation

by **Neel Nanda** 6th Jan 2023

This is the seventh post in a sequence called 200 Concrete Open Problems in Mechanistic Interpretability. [Start here](#)[°], then read in any order. If you want to learn the basics before you think about open problems, check out [my post on getting started](#). Look up jargon in [my Mechanistic Interpretability Explainer](#)

Motivating papers: [Causal Scrubbing](#)[°], [Logit Lens](#)[°]

Motivation

In Mechanistic Interpretability, the core goal is to form true beliefs about what's going on inside a network. The search space of possible circuits is *extremely* large, and even once a circuit is found, we need to verify that that's what's really going on. These are hard problems, and **having good techniques and tooling is essential to making progress**. This is particularly important in mech interp because it's such a young field that there isn't an established toolkit and standard of evidence, and each paper seems to use somewhat different and ad-hoc techniques (pre-paradigmatic, in Thomas Kuhn's language).,

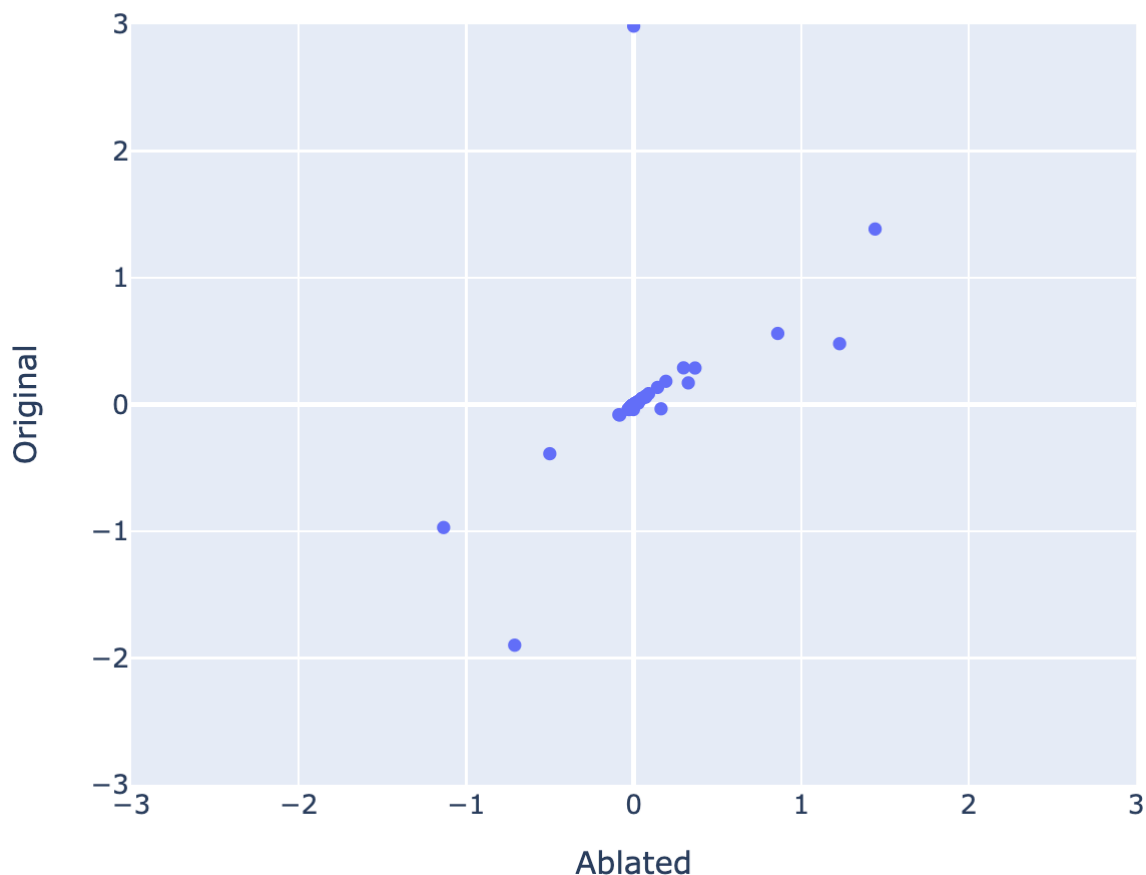
Getting better at this is particularly important to enable us to get traction interpreting circuits at all, even in a one layer toy language model! But it's particularly important for dealing with the **problem of scale**. Mech interp can be very labour intensive, and involve a lot of creativity and well honed research intuitions. This isn't the end of the world with small models, but ultimately we want to understand models with hundreds of billions to trillions of

parameters! We want to be able to leverage researcher time as much as possible, and the holy grail is to eventually automate the finding of circuits and understanding models. My guess is that the most realistic path to *really* understanding superhuman systems is to slowly automate more and more of the work with weaker systems, while making sure that we understand those systems, and that they're aligned with what we want.

This can be somewhat abstract, so here are my best guesses for what progress could look like:

- **Refining understanding:** In what contexts are current techniques most useful, and where are they misleading? How can we notice misapplications? Which techniques fail in the same way?
 - Eg, what's up with [backup heads](#), which take over when the main head is [ablated](#)
 - This graph shows the direct effect on the logits from each head in Indirect Object Identification, before and after ablating an important name mover head - the ablated head is the point at the top (important $\rightarrow 0$), but there are two heads that move significantly to compensate - the two other points far off the diagonal.

Original vs Post-Ablation Direct Logit Attribution of Heads



◦

- **How to find circuits?:** What are the right mindsets and approaches for finding a novel circuit? Common traps? What tools are best to apply in an unfamiliar situation, and how to interpret their output?
 - I expect this to look like a mix of a refined understanding of current techniques, building great infrastructure, and building practical intuitions and experience.
- **Building a better toolkit:** What kind of new, general techniques are out there for really understanding model internals? One of the joys of mech interp is that we have *full* control over the model's internals, and can edit whatever weights and activations we want. I expect there's many ways to use this that no one's tried!
 - [Causal tracing/activation patching](#) from [ROME](#) is a great example of this! It's such an elegant, powerful and generally applicable technique, that I'd just never thought of before
- **Gold standards of evidence:** What does it *mean* to have truly understood a circuit? Are there generally applicable approaches
 - Redwood's [Causal Scrubbing](#) is a solid attempt here, and I'm excited to see how well it works in practice
- **Good infrastructure:** The right software and tooling can *massively* increase the rate of being able to do research - if common operations can be done in a few lines of code and run quickly, you can focus on actually doing research.
 - My [TransformerLens library](#) has significantly increased my rate of doing research! Labs often have great internal infrastructure, but sadly this is rarely open source
- **Automation:** Taking existing work that a researcher would do, and either doing it faster or fully automating it. This could range from just saving a researcher time to the ambitious goal of automatically finding novel circuits.
 - A simple example is creating metrics that can identify [induction heads](#) in arbitrary models

A mechanistic perspective: This is an area of mech interp research where it's particularly useful to study non-mechanistic approaches! People have tried a lot of techniques to understand models. Approaches that solely focus on the model's inputs or outputs don't seem that relevant, but there's a lot of work digging into model internals ([Rauker et al](#) is a good survey) and many of these ideas are fairly general and scalable! I think it's likely that there are insights here that are underappreciated in the mech interp community.

A natural question is, what does mech interp have to add? I have a pretty skeptical prior on *any* claim about neural network internals, let alone that there's a technique that works in general or that can be automated without human judgement. In my opinion, one of the core

things missing is **grounding** - concrete examples of systems and circuits that are well understood, where we can test these techniques and see how well they work, their limitations, and whether they miss anything important. My vision for research here is to take circuits we understand, and use this as training data to figure out scalable techniques that work for those. And to then use these refined techniques to search for new circuits, do our best to fully understand *those*, and to build a feedback loop that validates how well these techniques generalise to new settings.

Further Thoughts

“Build better techniques” is a broad claim, and encompasses many approaches and types of techniques. Here’s my attempt to operationalise the key ways that techniques can vary - note that these are spectrums, not black and white! Important context is that I generally approach mech interp with two mindsets: **exploration**, where I’m trying to become less confused about a model and form hypotheses, and **verifying/falsifying**, where I’m trying to break the hypotheses I’ve formed and look for flaws, or for stronger evidence that I’m correct. Good research looks like regularly switching between these mindsets, but they need fairly different techniques.

1. General vs specific:

- a. **General** techniques are a broad toolkit that work for many circuits, including ones we haven’t identified yet.
 - i. Eg: [direct logit attribution](#) - looking at which model components directly contribute to the logit for the correct next token.
- b. **Specific** techniques focus on identifying a single type of circuit/circuit family
 - i. Eg: [Prefix matching](#), identifying induction heads by looking for the induction attention pattern on repeated random tokens
- c. All other things being the same, general techniques are much more exciting, but specific techniques can be much easier to create, and can still be very useful (it’s great that we can automatically identify all induction heads in a model!)

2. Exploratory vs confirmatory:

- a. **Exploratory** techniques are about getting information about confusing behaviour in a model - what can we do to learn more about model internals and get more data on what’s going on? These tend to be pretty general.
 - i. Visualising model internals is a good example, eg looking attention patterns, or plotting the first 2-3 principal components of activations.

b. **Confirmatory** techniques focus on taking a hypothesised circuit, and confirming or falsifying whether that's what's actually going on. Ideally something objective enough that other researchers can trust them. These can be either specific or general.

i. [Causal Scrubbing](#) is the best attempt I've seen here

c. This is emphatically a spectrum - good exploratory techniques also help verify circuits, and being able to quickly verify a circuit can significantly help exploration.

d. One key difference is how subjective vs objective the output is. Exploratory techniques can output high dimensional data and visuals for a researcher to subjectively interpret as they form hypotheses and iterate, while confirmatory techniques should give objective output, eg a specific metric of how good the circuit is.

3. **Rigorous vs suggestive:** Techniques vary from the rigorous, with strong and reliable evidence, to the merely suggestive.

a. **Rigorous** example: [Activation patching](#).

i. If copying a single activation from input A to input B is sufficient to flip it from answer B to answer A, you can be pretty confident that that activation contained the key information distinguishing A from B

b. **Suggestive** example: Interpreting a neuron by looking for patterns in its [max activating dataset examples](#).

i. This is [known to be misleading](#), and earlier parts of the neuron's activation range could easily mean other things. But, equally, it definitely tells you *something* useful about that neuron, and can be strong evidence that a neuron is *not* monosemantic!

c. This is emphatically a spectrum! No technique is perfect, and all of them have *some* pathological edge cases. Equally, even merely suggestive techniques can be useful for forming hypotheses and iterating.

i. In practice, my advice is to have a clear view of the strengths and weaknesses of each technique, and to take it as useful but limited data

ii. A particularly important thing is to track the *correlations* between technique failures. Activation patching may fail in similar ways to ablations, but likely max activating dataset examples

d. This can be unidirectional - if ablating a component kills model performance then that's decent evidence that it matters, but if that has no effect, there may be a backup head taking over

- i. Note that ablating a component may break things because it breaks *all* model behaviour (eg if it's used as an important bias term) - this happens with MLP0 in GPT-2 Small

4. **Scalable vs labour intensive:** Techniques vary from fully automated approaches that can be run on arbitrary models and produce a simple output, to painstakingly staring at neurons and looking at weights.

Finally, a cautionary note to **beware premature optimization**. A common reaction among people new to the field is to dismiss highly labour intensive approaches, and jump to techniques that are obviously scalable. This isn't crazy, scalable approaches are an important eventual goal! But if you skip over the stage of *really* understanding what's going on, it's very easy to trick yourself and produce techniques or results that don't really work.

Further, I think it is a mistake to discard promising seeming interpretability approaches for fear that they won't scale - there's a lot of fundamental work to do in getting to a point where we can even understand small toy models or specific circuits at all. I see a lot of the work to be done right now as being basic science - building an understanding of the basic principles of networks and a collection of concrete examples of circuits (like what is up with [superposition](#)?!), and I expect this to *then* be a good foundation to think about scaling. We only have like 3 examples of well understood circuits in real language models! It's plausible to me that we shouldn't be focusing too hard on automation or scalable techniques until we have at least 20 diverse example circuits, and can get some real confidence in what's going on!

But automated and scalable techniques remain a vital goal!

Tips

- There are many other forms of premature optimisation here to be aware of - putting a lot of effort into creating conceptually elegant techniques, or infrastructure that can be run at scale, when there's a fundamental flaw in the basic idea. Try to fail fast and get your hands dirty ASAP. I recommend starting with a concrete example(s) of the kind of behaviour you care about, and focusing on really understanding that example. And then trying to find faster and more efficient ways of replicating that understanding, and steadily scaling it up. But regularly testing on that example(s) to get fast feedback and debug.
- An easy initial way to validate techniques is by studying a circuit in depth in one model, and then trying it on another model in the same family. This works best for model

scans, ideally with models close together in size

- The [Stanford CRFM scan](#) has 5 identically sized GPT-2 Small and GPT-2 Medium models with different random seeds!
- In TransformerLens, all models are loaded into a consistent architecture, and the same code can run by just changing the string in `TransformerLens.from_pretrained`. This makes it much easier to see how ideas generalise across models/scales!
 - For example, here's [a mosaic I made](#) of the induction heads in 40 different models using identical code
- One major advantage of reverse engineering neural networks (over, eg, biology or neuroscience) is that we have *full control* over model internals, and know exactly what computation the model is doing (though not necessarily what it means!). This gives a lot of room to do careful, targetted, causal interventions.
- My guess is that the easiest way to implement techniques will be by building it on [my TransformerLens library](#), which has a bunch of infrastructure to make it easy to write code accessing and editing model internals, so you can iterate fast. See [a demo of techniques here](#).
 - If you're investigating causal tracing/activation patching, the ROME paper also provides code for this, but in my opinion TransformerLens will be easier.
- Problems that involve running a technique across a ton of data and looking for the best dataset examples will be more of an infrastructure pain than ones focusing on a handful of specific prompts.
 - I have some existing infrastructure to do this, feel free to [email me](#) if you want to do a project involving this

Resources

- The [techniques section](#) of my mech interp explainer - the main ones worth reading about are **direct logit attribution**, **activation patching** and **ablations**
 - And [the explanations of currently understood transformer circuits](#) - most notably **induction circuits** and **indirect object identification circuits**
- **Demo:** The [Exploratory Analysis Demo](#) for TransformerLens demonstrates using direct logit attribution, activation patching and ablations on the IOI task
- [Redwood's Causal Scrubbing](#)° sequence, my current favourite attempt to make a general, automated, confirmatory technique (and which hopefully works for exploration too!)

- There is not currently open source tooling for this, sadly
- Mechanistic Interpretability infrastructure for LLMs:
 - My [TransformerLens](#) library
 - Inspired by [Anthropic's Garcon](#)
 - [Alan Cooney's CircuitsVis](#) library, for integrating interactive visualizations in React with Python code
 - Nostalgebraist's [Transformer-Utills](#) library
 - My (under construction!) [Neuroscope website](#), for seeing the max activating dataset examples of neurons
- Other existing tooling for LLMs that may be relevant:
 - Google PAIR's [Learning Interpretability Tool](#)
 - [Captum](#), an interpretability library (I believe focused on feature attribution?)
- [Rauker et al's survey](#) of inner interpretability techniques (including a lot of non-mechanistic work)

Problems

[This spreadsheet](#) lists each problem in the sequence. You can write down your contact details if you're working on any of them and want collaborators, see any existing work or reach out to other people on there! (thanks to Jay Bailey for making it)

- **A-C* 6.1** - Breaking current techniques - trying to find concrete edge cases where the technique breaks.
 - I'd start by looking for a misleading example in a real model or training an algorithmic toy model with one, and then trying to distil out *why* it breaks and whether we could detect it.
 - **B* 6.2** - [Direct logit attribution](#) - I'd start by looking at GPT-Neo Small, where the logit lens (a precursor to direct logit attribution) seems to work badly, but to work well if you include the final layer *and* the unembed
 - **C* 6.3** - Can you *fix* direct logit attribution in GPT-Neo Small, eg by finding a linear approximation to the final layer by taking gradients?
 - Eleuther's tuned lens (see channel [#interp-across-depth](#) in their discord) project will be a good place to start.
 - Other ways it might fail: where a component matters by reducing incorrect logits (changing the logsumexp in log_softmax) or changing the final

layernorm scale.

- **B* 6.4 - Linearising LayerNorm** - see some cool work by [Eric Winsor at Conjecture°](#) on how LayerNorm can be used for computation
 - I'd start by looking at the scale factor for each layernorm across a bunch of data - they sometimes have bimodal distributions, which suggests it's doing something interesting!
- **B-C* 6.5 - Activation patching**
 - A key intuition is that activation patching will break when there's dependence on multiple variables and you only patch one, eg a head only has the right attention pattern if A AND B are true. This is why careful counterfactuals work best (where you keep the two prompts as similar as possible)
- **C* 6.6 - Causal scrubbing**
- A-B 6.7 - [Ablations](#) - I'd start with the backup name movers in the IOI Circuit, where we know that zero ablations break
 - Bonus: Look at mean ablation and random ablation too.
 - **B* 6.8** - Can you find places where one of these breaks but the others don't?
- B 6.9 - [Composition scores](#) - anecdotally, they don't work well for the IOI circuit.
- B 6.10 - [Eigenvalue copying score](#)
- **C* 6.11** - Automate ways to identify heads that compose. I'd start by looking at the IOI circuit and the heads that compose there and looking for metrics that can detect this.
 - A good place to start are the [composition scores](#) in [A Mathematical Framework](#) - anecdotally these don't seem to work for the IOI circuit, but may be a good starting point.
 - **B* 6.12** - Rather than analysing the weights, try looking for composition on a specific input. Decompose the residual stream into the sum of outputs of previous heads, and then decompose the query, key and value of the next head into sums of terms from each previous head. Are any of these terms larger than the others? Do any matter significantly more if you ablate them? Etc.
 - **C* 6.13** - Can you do this with [direct path patching](#), as used in the IOI paper?
- **B* 6.14** - Compare [causal tracing](#) (where you corrupt specific token embeddings by adding noise and patch in clean activations) to [activation patching](#) (where you corrupt the prompt by using a similar prompt with a different answer, eg replacing Eiffel Tower

with Colosseum). Do they give the same outputs? Can you find situations where one breaks and the other doesn't? I'd start by studying the IOI task or the factual recall task.

- In the [ROME paper](#) they do activation patching by patching over the outputs of 10 adjacent MLP or attention layers. (Note - I recommend looking at [logit difference](#) after patching, not absolute probabilities of the correct token)
 - **A* 6.15** - How do results change when you do single layers?
 - **A* 6.16** - Can you get anywhere when patching specific neurons?
 - **B* 6.17** - Can you get anywhere when patching some set of neurons (eg the neurons in the range of 10 layers that activate the most, or have the highest activations)
- Automated ways to find specific circuits
 - Automated ways to analyse attention patterns to find different kinds of heads (tip: for induction circuits, the easiest way is via feeding in repeated random tokens and looking at the average attention paid to the correct token for that head)
 - A 6.18 - [Previous token heads](#)
 - A 6.19 - [Duplicate token heads](#)
 - A 6.20 - [Induction heads](#)
 - **A* 6.21** - [Translation heads](#)
 - **B* 6.22** - [Few shot learning heads](#)
 - **B* 6.23** - Can you find an automated way to detect [pointer arithmetic](#) based induction heads vs classic induction heads?
 - **B* 6.24** - Detecting the heads used in the [IOI Circuit](#) (S-Inhibition, name mover, negative name mover, backup name mover)
 - **B* 6.25** - The heads used in [factual recall](#) to move information about the fact to the final token - I'd identify these via activation patching
 - **B-C* 6.26** - (Infrastructure) Combine some of the above head detectors to make a "wiki" for a range of models, with information and scores for each head for how much it falls into different categories?
 - The MVP here would just be a bunch of saved pandas Dataframes with a row for each head, and a column for each metric. And doing this by
 - **C* 6.27** - Can you do a similar thing for neuron interpretability? Eg finding trigram neurons

- **B-C* 6.28** - Finding good ways to find the equivalent of max activating dataset examples for attention heads. I'd validate it on induction circuits and then on the IOI circuit. Ideas:
 - Max norm of the result (the thing that head adds to the residual stream)
 - Max entropy of attention pattern
 - Max attention paid to any token (other than the first token)
 - Max norm of value (weighted by attention)
 - Max attention or entropy in the attention pattern weighted by the norm of the value vector at that source position.
- **B-C* 6.29** - Refining the max activating dataset examples LINK technique for neuron interpretability to eg find minimal or diverse examples. Some brainstormed thoughts:
 - **B* 6.30** - Corrupt different token embeddings in the sequence to see which matter (eg by adding Gaussian noise to the embedding, or replacing them with a random other token)
 - **B* 6.31** - Compare these to randomly chosen directions in neuron activation space, and see if there's a noticeable difference in how clustered/monosemantic things seem
 - **B* 6.32** - Validate these by comparing the max activating examples to the direct effect of that neuron on the logits or to the output vocab logits most boosted by that neuron.
 - Direct effect on the logits means taking `W_out[neuron_index, :] @ W_U``
 - **B-C* 6.33** - Using a model like RoBERTa or GPT-3 to find similar text to an existing example and seeing if these also activate the neuron.
 - Bonus: Use these models to replace specific tokens - this will be easiest with a model trained with the same tokenizer
 - **B-C* 6.34** - Look at dataset examples at different quantiles for the neuron activations (eg 25%, 50%, 75%, 90%, 95%). Does this change anything?
 - **B-C* 6.35** - (Infrastructure) Add any of the above to Neuroscope ([email me](#) for access to the codebase, I haven't made it clean enough to open source yet)
 - A 6.36 - Finding the minimal example to activate a neuron by truncating the text - how often does this work?
 - A 6.37 - Can you replicate the results of [the interpretability illusion](#) for my toy models, by finding seemingly monosemantic neurons on Python Code or C4 (web text), but which are polysemantic when you study both?

- B 6.38 - In SoLU models, compare the max activating results for the pre SoLU, post SoLU and post LayerNorm activations (called `pre`, `mid` and `post` in TransformerLens). How consistent are the results? Does one seem more principled?
- Using LLMs to interpret models - I don't have great ideas here, but I'm sure there's something!
 - **B* 6.39** - Can GPT-3 figure out trends in max activating examples for a neuron?
 - **B* 6.40** - Can you use GPT-3 to generate counterfactual prompts with lined up tokens to do activation patching on novel problems? (Eg "..., John gave a bottle of milk to -> Mary" vs "..., Mary gave a bottle of milk to -> John" for the IOI task)
 - D 6.41 - Choose your own adventure - can you find a way to usefully use an LLM here?
- Take techniques from the rest of interpretability, apply them on circuits we understand, and try to validate how well they work.
 - **B-C* 6.42** - Feature attribution
 - Integrated gradients seem one of the most principled and promising here - [the Captum library](#) seems to have an implementation
 - How does this compare to max activation dataset examples or feature attribution for MLP neurons?
 - **B-C* 6.43** - Probing
 - Can you get any evidence for or against the predictions in Toy Models of Superposition? (Eg correlated features are more orthogonal, or superposed features self-organise into different orthogonal subspaces - I'd look for antipodal pairs first)
 - **C* 6.44** - Pick anything else that seems interesting from [Rauker et al](#)
 - D 6.45 - [Wiles et al](#) gives an automated set of techniques to analyse bugs in image classification models, by using a text-to-image model to generate many synthetic images, clustering misclassified inputs, and using a separate model to analyse these. Can you get any traction in adapting this to language models?
- **C* 6.46** - Taking existing well understood circuits like induction heads or Indirect Object Identification and explore general quantitative ways to characterise that it's a true circuit (or trying to disprove that it's a well-understood circuit!)
 - [Redwood's causal scrubbing algorithm](#) is a great place to start
- **C* 6.47** - Build on [Arthur Conmy's work](#) to automatically find circuits via recursive path patching

- A-C 6.48 - Resolve some of the [open issues/feature requests](#) for TransformerLens
- **B-C* 6.49** - Build tooling to take the “diff” of two models, just treating them as a black box mapping inputs to outputs (so it works with two models with different internal structure, eg a 6 layer and 8 layer transformer of the same architecture).
 - A good validation will be taking the [1L and 2L attn-only models](#) and looking for techniques that identify the fact that induction heads are a really big deal in the 2L!
 - **B* 6.50** - Run them on a bunch of text and look at the biggest per-token log prob difference
 - What is the best metric here? Difference in prob, difference in log prob, difference in logit, difference in $\max(-5, \log \text{prob})$, etc all seem reasonable.
 - B 6.51 - Run them on various benchmarks and compare performance
 - Variant: Compare the per data point performance too and analyse outlier points.
 - **B* 6.52** - Try “benchmarks” of the ability to perform algorithmic tasks like IOI, acronyms, emails, etc as described in the circuits in the wild section
 - B 6.53 - Try qualitative exploration, like just generating text from the models given various prompts, and see if this sparks any interesting ideas
- **B-C* 6.54** - Build tooling to take the “diff” of two models with the *same* internal structure (eg models trained on different random seeds or different checkpoints). This includes all of the above, but also lets you directly compare model internals!
 - B 6.55 - Look at the difference in weights, and look for the largest difference
 - B 6.56 - Run them on a bunch of text and compare the activations - look for biggest differences
 - Attention patterns may be particularly interesting to compare
 - **B* 6.57** - Look at the direct logit attribution of layers and heads on various texts - for each component, look for the biggest differences across a bunch of text.
 - **B* 6.58** - Do activation patching on a piece of text where one model does much better than the other - are there any parts that are key to the improved performance?
- **B-C* 6.59** - We can understand how attention is calculated for a head using the [QK matrix](#). This doesn't work for [rotary attention](#), since different relative positions have a different matrix, can you find a principled alternative?
 - I'd start by analysing a previous token head or induction head trained in a toy algorithmic model, and then by looking at induction heads in a real rotary model.

`pythia-19m` is the smallest available in TransformerLens.

Previous:

200 COP in MI: Analysing Training Dynamics

No comments 16 karma

Next:

200 COP in MI: Image Model Interpretability

1 comments 18 karma

[Log in to save where you left off](#)

Mentioned in

- 41 200 Concrete Open Problems in Mechanistic Interpretability: Introduction
- 25 AXRP Episode 19 - Mechanistic Interpretability with Neel Nanda
- 23 Attribution Patching: Activation Patching At Industrial Scale
- 14 Mechanistic Interpretability Quickstart Guide

[Moderation Log](#)

More from Neel Nanda

19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0
32	Models May Behave Worse When Eval Aware...	Senthooran Rajamanoharan, Neel ...	2d	0
65	models have some pretty funny attractor sta...	aryaj, Senthooran Rajamanoharan, ...	3mo	0

View more

Curated and popular this week

39	Sympathy for both sides of the egregious misalignment debate	Steven Byrnes	7h	5
26	A Mike’s-Eye View of ARC’s Research	Mikewins	3d	0
19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0